DeepWiki Prompt Customization Guide

All prompts live at [src/OpenDeepWiki/prompts/](file:///Users/jcachat/code_sandbox/antigravity-deepwiki/src/OpenDeepWiki/prompts).

Master Summary Table

Prompt File	Pipeline Phase	Size	Role Given to AI	Output Tool	Key Deliverable
[catalog-generator.md](file:///Users/jcachat/code_sandbox/antigravity-deepwiki/src/OpenDeepWiki/prompts/catalog-generator.md)	📋 Catalog Generation	168 lines	"Senior code repository analyst"	`WriteCatalog(json)`	Wiki table of contents (JSON tree)
[content-generator.md](file:///Users/jcachat/code_sandbox/antigravity-deepwiki/src/OpenDeepWiki/prompts/content-generator.md)	📝 Content Generation	1046 lines	"Professional technical documentation writer and code analyst"	`WriteDoc(content)`	Full Markdown wiki page per catalog item
[mindmap-generator.md](file:///Users/jcachat/code_sandbox/antigravity-deepwiki/src/OpenDeepWiki/prompts/mindmap-generator.md)	🧠 Mind Map	194 lines	"Senior software architect"	`WriteMindMap(content)`	Hierarchical architecture mind map
[incremental-updater.md](file:///Users/jcachat/code_sandbox/antigravity-deepwiki/src/OpenDeepWiki/prompts/incremental-updater.md)	🔄 Incremental Update	910 lines	"Documentation maintenance specialist"	`DocTool.EditAsync/WriteAsync`	Targeted doc updates from code diffs

Template Variables (auto-injected at runtime)

Variable	Used In	Source
`{{repository_name}}`	All 4 prompts	Repository name from DB
`{{language}}`	All 4 prompts	Language code ([en](file:///Users/jcachat/code_sandbox/antigravity-deepwiki/src/OpenDeepWiki/Services/Chat/ChatAssistantService.cs#151-156), `zh`, etc.)
`{{project_type}}`	catalog, mindmap	Detected primary language (C#, Python, etc.)
`{{key_files}}`	catalog, mindmap	Auto-detected entry points
`{{entry_points}}`	catalog, mindmap	Main bootstrapping files
`{{file_tree}}`	catalog, mindmap	Directory tree in TOON format
`{{readme_content}}`	catalog, mindmap	Contents of README.md

Variable	Used In	Source
`{{git_url}}`	content	Git remote URL
`{{branch}}`	content	Branch being analyzed
`{{file_base_url}}`	content	Base URL for source links
`{{catalog_path}}`	content	Path of current catalog item
`{{catalog_title}}`	content	Title of current catalog item
`{{previous_commit}}`	incremental	Old commit SHA
`{{current_commit}}`	incremental	New commit SHA
`{{changed_files}}`	incremental	List of files changed between commits

Per-Prompt Deep Dive

1. 🗂️ Catalog Generator

What it does: Reads the repo structure, entry points, and README, then generates a JSON catalog tree (wiki table of contents).

Tools available to AI:

Tool	Purpose
`ListFiles(glob, maxResults)`	Discover relevant source files
[ReadFile(path)](file:///Users/jcachat/code_sandbox/antigravity-deepwiki/src/OpenDeepWiki/MCP/McpRepositoryTools.cs#211-248)	Read entry points and key files
`Grep(pattern, glob)`	Find patterns across codebase
`WriteCatalog(json)`	Output the final catalog (required)

Customizable sections you can edit:

Section (line range)	What to customize	Example change
Role (L3-5)	Change the AI's persona	Add domain expertise, e.g., "You specialize in data engineering pipelines"
Design Principles (L45-62)	Granularity and grouping rules	Change from "business domain" to "data pipeline stages"
Standard Catalog Template (L129-142)	Default sections to include	Add "Data Lineage", "DAG Structure", remove "API Reference"
Workflow Steps (L68-107)	Discovery patterns	Add custom grep patterns for your codebase
Anti-Patterns (L156-163)	What to avoid	Add domain-specific anti-patterns

Output format:

```
{
  "items": [
    { "title": "Title (in {{language}})", "path": "lowercase-hyphen-path", "order": 0, "children": [] }
  ]
}
```

2. 📝 Content Generator

What it does: For each catalog item, reads relevant source files and generates comprehensive Markdown documentation with Mermaid diagrams, code examples with source attribution, and API references.

Tools available to AI:

Tool	Purpose
[ReadFile(relativePath, offset?, limit?)](file:///Users/jcachat/code_sandbox/antigravity-deepwiki/src/OpenDeepWiki/MCP/McpRepositoryTools.cs#211-248)	Read source files (with pagination)
`ListFiles(glob, maxResults)`	Find relevant files
`Grep(pattern, glob, caseSensitive?, contextLines?, maxResults?)`	Search codebase
`WriteDoc(content)`	Save the final Markdown page
[EditDoc(oldContent, newContent)](file:///Users/jcachat/code_sandbox/antigravity-deepwiki/web/app/admin/repositories/%5Bid%5D/page.tsx#664-672)	Patch existing docs
`ReadDoc()`	Read current doc content
`DocExists()`	Check if doc exists

Mandatory 3-phase process:

1. **GATHER** — Discover and read source files (P0→P4 priority)
2. **THINK** — Analyze patterns, map relationships, design diagrams
3. **WRITE** — Compose document following the template

Customizable sections you can edit:

Section (line range)	What to customize	Example change
System Constraints (L1-48)	Core rules the AI must follow	Relax "no fabrication" for internal docs, or add stricter rules
Role Definition (L52-63)	AI persona and capabilities	Add "expertise in Airflow DAGs and BigQuery"
Documentation Principles (L209-217)	What makes good docs	Emphasize "operational runbooks" over "design intent"
Document Structure Template (L400-494)	Required sections in every doc	Add "Troubleshooting", "SLA", remove "API Reference"
Section Requirements (L496-509)	Which sections are mandatory/conditional	Make "Configuration" always required
Mermaid Diagram Requirements (L537-743)	Diagram types and templates	Add custom diagram templates for your architecture
Quality Checklist (L814-887)	Verification gates	Add checks for "operational accuracy"
Multi-language Support (L890-935)	Language-specific formatting	Add custom language rules

3. 🧠 Mind Map Generator

What it does: Creates a hierarchical architecture mind map using `#` / `##` / `###` headings linked to source paths.

Tools available to AI:

Tool	Purpose
`ListFiles(glob, maxResults)`	Discover project structure
[ReadFile(path)](file:///Users/jcachat/code_sandbox/antigravity-deepwiki/src/OpenDeepWiki/MCP/McpRepositoryTools.cs#211-248)	Read entry points
`Grep(pattern, glob)`	Find main modules
`WriteMindMap(content)`	Output the mind map (required)

Customizable sections you can edit:

Section (line range)	What to customize	Example change
Role (L3-5)	AI persona	"You specialize in data pipeline architecture"
Mind Map Format (L44-65)	Hierarchy format and max depth	Change max depth from 3 to 4
Structure Guidelines (L68-104)	Templates per project type	Add "For Data Pipeline Projects (Airflow, Spark)" template
Workflow (L108-147)	How the AI explores	Add custom discovery patterns for DAGs
Anti-Patterns (L160-167)	What to avoid	"Don't list individual DAG tasks"

Output format:

```
# Level 1 Topic
## Level 2 Topic:path/to/file
### Level 3 Topic
```

4. 🔄 Incremental Updater

What it does: Analyzes git diffs between commits and surgically updates only the affected wiki pages.

Tools available to AI:

Tool	Purpose
`GitTool.ListFiles(pattern?)`	List repo files
`GitTool.Read(path)`	Read changed files
`GitTool.Grep(pattern, glob?)`	Search for references
`CatalogTool.ReadAsync()`	Get current wiki catalog
`CatalogTool.WriteAsync(json)`	Replace catalog (major changes)
`CatalogTool.EditAsync(path, json)`	Edit single catalog node
`DocTool.ReadAsync(path)`	Read existing doc
`DocTool.WriteAsync(path, content)`	Rewrite entire doc
`DocTool.EditAsync(path, old, new)`	Targeted find/replace in doc

Customizable sections you can edit:

Section (line range)	What to customize	Example change
System Constraints (L5-47)	Rules for update behavior	"Always preserve change history in docs"
Change Categories (L318-328)	Priority mapping	Elevate "Configuration Changes" to High priority
Execution Steps (L332-401)	Update workflow	Add step for "notify team on breaking changes"
Quality Checklist (L571-610)	Verification requirements	Add "Verify DAG documentation matches current graph"
Skip Conditions (L879-886)	When NOT to update docs	"Also skip changes to migration scripts"

How to Customize

1. **Edit the [.md](file:///Users/jcachat/code_sandbox/Gemini-Cookbooks/README.md) files directly** in [src/OpenDeepWiki/prompts/](file:///Users/jcachat/code_sandbox/antigravity-deepwiki/src/OpenDeepWiki/prompts/) — they're plain Markdown with `{{variable}}` template placeholders
2. **Rebuild the Docker image** — prompts are baked into the image at build time
3. **Restart processing** — existing repos need to be re-queued (use "Regenerate" button) to use updated prompts

Tip

The most impactful customization for your use case would be editing the **Catalog Generator's Design Principles** and **Standard Catalog Template** to match your domain (e.g., Airflow DAGs, BigQuery pipelines, SOW workflows).

Important

Template variables like `{{repository_name}}` are injected at runtime by the backend code in [WikiGenerator.cs](file:///Users/jcachat/code_sandbox/antigravity-deepwiki/src/OpenDeepWiki/Services/Wiki/WikiGenerator.cs). Do not rename or remove them without updating the C# code.